TP-3 : Les objets et tableaux en JS

TP JavaScript « pur »

Ce TP a pour but manipuler les objets et tableaux en JavaScript (créer, modifier, parcourir, fusionner et traiter des structures de données complexes).

✍ *Exercice 1 – (Créer et manipuler des objets simples)*

1. Créez un objet etudiant contenant les propriétés suivantes :
   - prenom ⟶ "Sara"
   - nom ⟶ "Benali"
   - age ⟶ 22
   - ville ⟶ "Casablanca"
   - Affichez ensuite chaque propriété dans la console avec console.log().
   - Utilisez la notation par point etudiant.nom et la notation par crochets etudiant["nom"].
2. Ajoutez une méthode parler() à l'objet, qui affiche dans la console :

```
'Bonjour, je m'appelle Sara Benali.'
```

   - Utilisez les *backticks* : `…` **et a**ppelez cette méthode et observez le résultat.
3. Créez un autre objet voiture avec les propriétés :
   - marque
   - modele
   - annee
   - proprietaire (qui est un objet avec nom et age)
   - Affichez ensuite : "La voiture Tesla Model 3 appartient à Anass, 23 ans."

✍ *Exercice 2 – (Vérifier, modifier et parcourir un objet)*

1. Dans l'objet etudiant
   - Vérifiez si la propriété "age" existe avec l'opérateur in.
   - Ajoutez une nouvelle propriété note: 15.
   - Modifiez la valeur de ville en "Rabat".
   - Supprimez la propriété note.
2. Parcourez toutes les propriétés de etudiant à l'aide d'une boucle for...in
3. Améliorez la boucle précédente :
   - affichez uniquement les propriétés réellement possédées par l'objet (et non héritées du prototype) à l'aide de hasOwnProperty()

✍ *Exercice 3 – (Fusionner et copier des objets)*

1. Créez deux objets :

```
const infos = { nom: "Ali", age: 25 };
const contact = { email: "ali@gmail.com", ville: "Fès" };
```

2. Fusionnez ces deux objets dans un nouvel objet profil :
   - d'abord avec Object.assign()
   - puis avec le spread operator (...).
3. Ajoutez une nouvelle propriété pays: "Maroc" à profil sans modifier les objets originaux.
4. Créez deux copies d'un objet user (de votre choix) :

```
const copie1 = Object.assign({}, user);
const copie2 = structuredClone(user);
```

- Modifiez ensuite :
  ```
  copie1.adresse.ville = "Agadir";
  ```

5. Affichez les trois objets (user, copie1, copie2) et expliquez :
   - Pourquoi user.adresse.ville a été modifié dans un cas, mais pas dans l'autre ?
   - Quelle est la différence entre copie superficielle (shallow copy) et copie profonde (deep copy) ?

✍ *Exercice 4 – (Tableaux et opérations de base)*

1. Créez le tableau suivant :
   ```
   const notes = [17, 19, 15, 20, 18, 16];
   ```
   - Affichez :
     - la première et la dernière note,
     - le nombre total d'éléments (length).
2. Modifiez le tableau :
   - Ajoutez une note avec push().
   - Supprimez la dernière avec pop().
3. Calculez la moyenne de toutes les notes avec :
   - une boucle for classique,
   - une boucle for...of,
   - la méthode forEach(),
   - la méthode moderne reduce().
4. Trouvez la note maximale et minimale à l'aide de reduce().

✍ *Exercice 5 – (Tableaux d'objets et méthodes modernes)*

1. Créez le tableau d'objets suivant :
   ```
   const etudiants = [
       { prenom: "Sara", note: 13 },
       { prenom: "Ali", note: 18 },
       { prenom: "Amin", note: 15 },
       { prenom: "Anas", note: 12 },
       { prenom: "Lina", note: 19 },
       { prenom: "Rachid", note: 9}
     ];
   ```
2. Affichez la moyenne générale des notes avec la combinaison de map() + reduce()
3. Trouvez : l'étudiant ayant la meilleure note (reduce() ou sort())
4. Ajoutez à chaque étudiant une propriété mention :
   - ≥16 → "Très bien"
   - ≥14 → "Bien"
   - ≥12 → "Assez bien"
   - sinon → "Insuffisant"
5. Créez un nouveau tableau contenant uniquement les étudiants ayant la mention "Bien" ou plus.

✍ *Exercice 6 – (Manipulations avancées sur tableaux d'objets)*

À partir du même tableau etudiants :

1. Filtrez les étudiants ayant une note ≥ 12 (filter()).
2. Affichez leurs prénoms en majuscules (map()).
3. Triez les étudiants par note décroissante (sort()). « ne pas muter l'original »
4. Cherchez le premier étudiant ayant exactement la note 12 (find()).